VARIBO: Generate-and-Explore for Learned Cross-Platform Query Optimization

Juri Petersen
IT University of Copenhagen
Copenhagen, Denmark
jurp@itu.dk

Mads Sejer Pedersen
IT University of Copenhagen
Copenhagen, Denmark
sepe@itu.dk

Zoi Kaoudi
IT University of Copenhagen
Copenhagen, Denmark
zoka@itu.dk

***Abstract*—Cross-platform data analytics frameworks execute complex dataflows across heterogeneous processing engines. A central problem in this setting is determining which engine executes each operator so that the dataflow runs efficiently. Existing cost-based optimizers require extensive manual tuning and struggle to capture nonlinear system behavior. Approaches based on learned cost models require eagerly collecting large training datasets and accurate runtime prediction, which is difficult to achieve. Both enumerate plans in a search space that grows exponentially with the number of operators and platforms. In this paper, we first formulate the problem of platform selection in cross-platform data processing as a structured multiclass prediction problem and propose VARIBO, a novel query optimization framework which pairs an online prediction phase with an offline generate-and-explore phase. Online, VARIBO employs a tree-convolutional classifier to assign a platform to every operator and produce an execution plan directly, avoiding expensive plan enumeration. Offline, VARIBO integrates a conditional, attribute-regularized β-variational autoencoder with latent-space Bayesian optimization, exploring alternative plans, executing the most promising ones, and retraining the classifier on the best discovered plans. We implement VARIBO in Apache Wayang and evaluate it on three benchmarks. VARIBO speeds up the total runtime of the test queries by 1.45–$2.20\times$ compared to Wayang's cost-based optimizer and halves the optimization time.**

I. INTRODUCTION

Modern data analytics increasingly spans multiple processing engines, such as relational databases, distributed dataflow systems, and embedded analytical engines, each of which excels at different operations. Executing a query on a single engine either requires moving all relevant data to that engine or can be substantially slower than combining engines within the query. A recent study [1] shows that, with the best choice of split point, executing analytical queries as two fragments on two different engines, including the cost of transferring the intermediate result, speeds up 40% of the queries by up to $14.87\times$ and reduces total workload time by 25–33% compared to the fastest single engine. The same study also shows that poorly chosen splits can be orders of magnitude slower. Combining engines within a query thus pays off, but only if the decision of which engine executes which part of the query is made well.

This decision is the core of *cross-platform query optimization* [2]–[5]: given a query plan with platform-agnostic operators and a set of processing platforms (engines), assign each operator to a platform so that the resulting execution plan minimizes a target metric, such as runtime. The problem differs from related settings that also involve multiple systems. In federated databases [6] and cross-database systems [7], [8], the underlying systems act as data sources: the data stays where it is, and query processing is organized around this placement for example with a mediator executing cross-source joins [6]. Polystores [9] integrate engines with different data models, but optimize queries mainly within groups of engines that share a data model. Query routing systems [10], [11] choose among interchangeable engines, but send each query as a whole to a single engine. In cross-platform query optimization, in contrast, the platforms are processing engines that can execute many of the same operators on the same data, and each operator can run on a different one. This makes the problem hard: the number of possible assignments grows exponentially with the number of operators, the engines exhibit heterogeneous and non-linear performance characteristics, and every switch between engines incurs a data movement cost.

Existing cross-platform optimizers fall short on these challenges. Cost-based cross-platform optimizers [4], [5], [12] rely on manually written, linear cost functions, which fail to capture non-linear engine behavior and the cost of moving data between engines. Learned query optimizers [13]–[19] have enhanced traditional optimizers in single-engine settings, but do not transfer directly. Hint-based approaches [15], [17], [18] steer the native optimizer of a single database system, a mechanism without a counterpart in a cross-platform setting. Learned cost models require the runtimes of many executed plans, which are very expensive to collect in a single engine [20], [21], let alone across heterogeneous engines. In addition, their accuracy suffers when plans span engines with different performance characteristics. Learning-to-rank optimizers [13], [17], [22] avoid predicting exact runtimes, but still require executed plans to obtain their training labels. For cross-platform systems, the learned optimizer of [23] replaces the cost model with a regression model and thus inherits both the cost of training data collection and the difficulty of runtime prediction. Moreover, most of these approaches share a common direction: they enumerate candidate plans and score them, which in the cross-platform setting means searching a space that grows exponentially with the plan size.

In this paper, we take a different approach. We observe that

the decision a cross-platform optimizer ultimately makes is a platform for each operator. Thus, we formulate cross-platform query optimization as a structured multiclass classification problem: instead of estimating the cost of candidate plans, we directly predict the platform of every operator in a single pass. This formulation offers two advantages. First, it avoids plan enumeration at query time. Second, a classifier only needs examples of good execution plans for training, whereas cost and ranking models must learn from the runtimes of both good and bad plans. An existing cost-based optimizer can therefore provide the initial training data without executing any plan.

We present VARIBO, a cross-platform query optimizer that builds on this formulation. Since a classifier can only be as good as its training examples, VARIBO pairs an online classifier with an offline generate-and-explore loop that discovers better plans than those of the initial optimizer. Online, a tree-convolutional classifier predicts the platform of each operator and produces an execution plan without enumerating any alternatives. Offline and in the background, a conditional attribute-regularized β-VARIational autoencoder learns a latent space of execution plans that is ordered by latency. In addition, a latent-space Bayesian optimization (BO) process navigates this space to generate and execute alternative plans. The best plans found are used to retrain the classifier. This loop exploits a key property of production analytical workloads: they are highly repetitive [24]. Exploring a recurring query once benefits all its later executions, and retraining can carry the improvements over to queries that were never explored. VARIBO decides where each operator runs, not how each engine executes it; data movement between engines and engine-level optimization are left to the underlying cross-platform system. We instantiate VARIBO in Apache Wayang[1]. However, VARIBO can be applied to any cross-platform system that exposes an operator-level plan representation and provides an initial set of plans to bootstrap from, such as those of an existing optimizer.

In summary, we make the following contributions:

- We formulate cross-platform query optimization as a structured multiclass classification problem, which avoids plan enumeration at query time and requires only examples of good plans for training. (Section II)
- We propose an optimizer architecture that pairs an online classifier with an offline generate-and-explore loop, which discovers better plans and retrains the classifier over time. (Section III)
- We devise a tree-based plan encoding that can represent both platform-agnostic and execution plans, and a tree-convolutional classifier with validation rules that guarantee executable plans. (Section IV)
- We pair a conditional attribute-regularized β-variational autoencoder, whose latent space is ordered by latency, with latent-space Bayesian optimization to generate and explore alternative better plans. (Section V)
- We implement VARIBO in an open-source cross-platform system, Apache Wayang, and evaluate it against its cost-based optimizer, a learned cost model, and single-platform assignments on three benchmarks, for both explored and unexplored queries. (Section VI)

We conclude the paper with related work (Section VII) and a summary of our findings (Section VIII).

[1]https://wayang.apache.org/

II. PROBLEM FORMULATION

We first formalize the problem of cross-platform query optimization as structured classification.

Cross-platform query optimization. Given an input plan where none of the operators is tied to a specific platform (platform-agnostic plan), the main goal of cross-platform query optimization is to find in which data system each operator has to be executed (execution plan), so that the plan runs most efficiently with respect to an optimization metric.

Platform-agnostic plan. We define a *platform-agnostic plan* as a directed acyclic graph (DAG) where the vertices are *platform-agnostic* operators (e.g., `Map`, `Filter`, `Join`), independent of the underlying system, and the edges represent the data flowing between the operators. Formally:

Definition 1 (Platform-agnostic plan). A platform-agnostic plan is a DAG $G = (O, E)$ where $O = \{o_1, o_2, \ldots, o_n\}$ is the set of platform-agnostic operators and $E \subseteq O \times O$ represents directed edges that form no cycle and capture data flowing between two operators.

Execution plan. We define an *execution plan* as a DAG where the vertices are *platform-specific* execution operators (e.g., `SparkMap`, `PostgresFilter`, `SparkJoin`), i.e., concrete implementations of an operator in a specific system, and the edges represent the data flowing between these operators. Formally:

Definition 2 (Execution plan). Let $P = \{p_1, p_2, \ldots, p_k\}$ be the set of available platforms. Given a platform-agnostic plan $G = (O, E)$, an execution plan is a DAG $G_{exec} = (O_e, E_e)$ where O_e contains, for each $o_i \in O$, the execution operator o_i^p that executes o_i on platform $p \in P$, and $E_e \subseteq O_e \times O_e$ is a set of directed edges that form no cycle and capture the data flowing between execution operators.

Cross-platform query optimization as a multiclass prediction problem. Most approaches treat cross-platform query optimization as classical query optimization, with a cost model estimating the cost of each candidate plan. The decision these optimizers ultimately make, however, is *per-operator platform selection*: assigning each operator of a platform-agnostic plan to a platform so as to produce an efficient execution plan. We formulate this problem as a multiclass classification problem with structured output. Structured output learning is a supervised ML approach that involves predicting values for structured objects instead of a single discrete or continuous value [25]. Unlike traditional classification tasks, which aim to model a many-to-one function, structured prediction models are more complex as they must account for the interrelations between the predicted variables. In our setting, the underlying

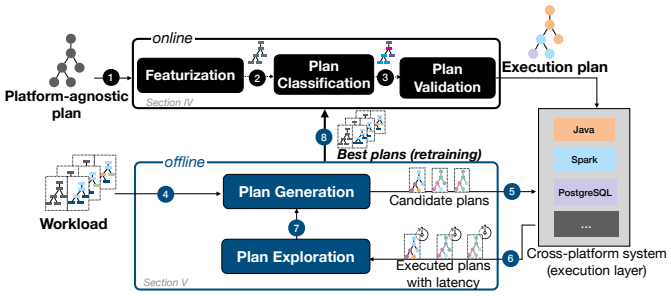


Fig. 1: Architecture of VARIBO.

data processing platforms constitute the labels used for prediction. Given a platform-agnostic plan, the objective is to classify each operator in the plan with the appropriate label so that the runtime of the resulting execution plan is minimized. This output is inherently structured: the choice made for one operator affects the others, since a platform switch between adjacent operators incurs a movement cost. Predicting each operator's platform independently is thus unsuitable, as it ignores these interdependencies. We formally define the problem:

Problem statement. *Let $G = (O, E)$ be a platform-agnostic plan and $P = \{p_1, p_2, \ldots, p_k\}$ be the set of available platforms on which the operators can be executed. The goal is to find a model $f : O \to P$ that selects a platform $p_i \in P$ for each operator $o \in O$ in G such that the total cost C of the resulting execution plan $G_{exec} = (O_e, E_e)$ is minimized:* $\min_f C(G_{exec})$.

C is a user-defined metric that may represent runtime, monetary cost, or energy consumption. In this paper, we focus on runtime. Note that whenever two adjacent operators are assigned to different platforms, data has to be moved from one platform to the other. How this movement is performed is out of the scope of this paper and should be handled by the underlying cross-platform system.

Given a platform-agnostic plan $G = (O, E)$ and the set of supported platforms P, the search space of all possible execution plans is of size $\mathcal{O}(|P|^{|O|})$. Since many operators have implementations on multiple platforms, the actual number of plans approaches this bound, making exhaustive enumeration impractical even for moderately sized plans.

III. VARIBO ARCHITECTURE

We propose VARIBO, a learned cross-platform query optimizer that replaces the traditional enumerate-and-prune approach with an online prediction phase and an offline generate-and-explore phase. Figure 1 shows the architecture comprising these online and offline components.

Online phase. At query optimization time, the online phase *predicts* the platform of each operator of a platform-agnostic plan. The plan is first featurized (❶) and given to a classifier (❷), which assigns a platform to every operator and thereby produces an execution plan without enumerating alternatives. A validation step then replaces assignments to platforms that do not support an operator (❸), and the resulting execution plan is passed to the cross-platform system for execution. This approach without any plan enumeration substantially reduces optimization time. As the classifier learns only which platform each operator should run on, it can be bootstrapped from the plans of an existing optimizer, i.e., from ⟨platform-agnostic plan, execution plan⟩ pairs, without runtime labels. This avoids the expensive training data collection of prior learned approaches, which *eagerly* execute many plans across multiple engines before the optimizer can be used.

Offline phase. The offline phase aims to discover better plans than those the classifier produces. Using a *generate-and-explore* strategy, it consists of two components: a *plan generator*, which produces candidate execution plans for a given platform-agnostic plan, and a *plan exploration* strategy, which uses the observed latencies of the candidates to decide which plans to generate next. For each platform-agnostic plan of the workload (❹), the generator produces candidate execution plans, which are executed on the cross-platform system (❺). Their latencies are passed to the exploration strategy (❻), which steers the generator toward more promising plans (❼). This loop repeats until a stopping condition is met. VARIBO thus collects runtime information *lazily*: it executes only the candidates that exploration finds promising, rather than labeling a fixed set of training plans upfront. The best plans found are then used to retrain the classifier (❽), which carries the improvements over to the explored queries and to similar queries that were never explored. Like the classifier, the generator is bootstrapped from the existing optimizer without executing any plan.

This separation keeps online optimization cheap: exploration runs offline, outside the critical path of query optimization, and can be scheduled when resources are available, e.g., during periods of low load. We describe the design of these components in Sections IV and V.

IV. ONLINE PLAN PREDICTION

At query optimization time, VARIBO translates a platform-agnostic plan into an execution plan in a single pass, without enumerating alternative plans. This section first describes how a plan is encoded into a tree of feature vectors (Section IV-A), then how the classifier predicts a platform for each operator from this encoding (Section IV-B), and how the validation rules guarantee an executable plan (Section IV-C).

A. Plan Featurization

Both the classifier and the generative model operate on a unified representation of platform-agnostic plans and execution plans, from which we must be able to reconstruct execution plans. In contrast to a previous learned approach, which uses a flat vector representation [23], we follow tree-based plan encodings [15], [16] and represent a plan as a tree in which each node is a vector describing an operator and its features, so that the structure of the plan is preserved.

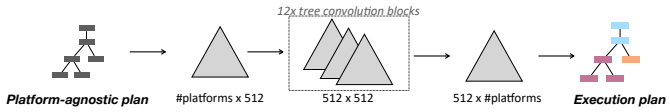


Fig. 2: VARIBO classifier architecture.

Each operator vector starts with a unique identifier, which only serves to reconstruct the execution plan from the predicted platforms and is removed before the vector is passed to the model. We encode the type of an operator, e.g., `Join` or `Filter` and the platform assigned to it in the execution plan using one-hot encodings. For platform-agnostic plans, all platform entries are set to 0, which distinguishes them from execution plans. To ensure a consistent ordering, both the operator types and the platforms are sorted alphabetically. Additionally, as operators in dataflow systems are often parameterized with user-defined functions (UDFs), whose complexity can significantly affect the execution time of an operator, we annotate each operator with the complexity class of its UDF: logarithmic, linear, quadratic, or superquadratic, encoded as a one-hot vector. We assume that the complexity class is provided as an annotation, e.g., by the developer of the UDF. Alternatively, it could be derived automatically by static analysis of the UDF code, combining techniques for inferring complexity bounds [26] with the code analysis of UDFs in dataflow systems [27], which we leave for future work. Finally, the last two entries contain the sum of the input cardinalities and the output cardinality of the operator, which capture the volume of data flowing through it. We obtain them from the cardinality estimates of an existing cost-based optimizer. In summary, each operator o is represented by a vector $\mathbf{x}_o$:

$$\mathbf{x}_o = \left[\, \mathrm{id}(o) \,\|\, \mathbf{t}(o) \,\|\, \mathbf{p}(o) \,\|\, \mathbf{u}(o) \,\|\, c_{\text{in}}(o) \,\|\, c_{\text{out}}(o) \,\right],$$

where $\mathrm{id}(o)$ is the identifier used for reconstruction, $\mathbf{t}(o)$, $\mathbf{p}(o)$, and $\mathbf{u}(o)$ are the one-hot encodings of the operator type, the assigned platform, and the UDF complexity class, and $c_{\text{in}}(o)$ and $c_{\text{out}}(o)$ are its total input and output cardinalities.

Unlike the featurization proposed in [23], which explicitly encodes the communication cost of data movement operators, we deliberately omit these operators from the encoding. Instead, the model accounts for the cost of moving data implicitly, by learning from pairs of platform-agnostic plans and good execution plans, in which platform switches occur only where they pay off. This keeps the feature set fixed and the encoding simple, as data movement operators would otherwise introduce input and output sizes that vary with the platform assignment. After prediction, we rely on the underlying cross-platform system to insert the necessary data movement operators at platform switches [28].

B. Plan Classification

Following our problem formulation, the online component of VARIBO comprises a classifier based on tree convolutions [16], [29]. Rather than enumerating candidate execution plans and estimating their costs, it directly predicts a platform for each operator of a featurized platform-agnostic plan, reducing query optimization to a classification task. Figure 2 shows its architecture, which consists of twelve tree convolution blocks, each comprising a binary tree convolution layer, a tree normalization layer, and a tree activation layer (the three stacked triangles in Figure 2). Tree convolutions capture the structural relations between operators by propagating information bottom-up through the plan, so that the representation of each operator is informed by its subtree rather than by the operator in isolation. For each operator, the classifier outputs a vector of logits, i.e., unnormalized scores, over the available platforms. The platform with the highest logit is selected, yielding an execution plan in a single forward pass.

C. Plan Validation

The predicted platforms do not always yield an executable plan, as not every operator is available on every platform (e.g., an operator with a user-defined function written in Java cannot be executed in PostgreSQL), a constraint that the featurization cannot capture. VARIBO therefore applies a set of lightweight validation rules to the output of the classifier before the plan is passed to the cross-platform system. When the predicted platform of an operator does not support it, VARIBO falls back to the most probable supported platform according to the logits. This is equivalent to masking unsupported platforms before selecting the platform with the highest logit. These rules are applied only during online prediction.

V. OFFLINE PLAN GENERATION AND EXPLORATION

The online classifier is only as good as the plans it is trained on, and the plans of a cost-based optimizer leave considerable room for improvement, as its cost formulas fail to capture the non-linear behavior of the engines and the cost of moving data between them. To discover better plans, we instantiate the plan generator of the offline phase with a variational autoencoder (VAE, Section V-A) and plan exploration with latent-space Bayesian optimization (LSBO, Section V-B). Figure 3 shows the resulting loop. The VAE encoder maps the ⟨platform-agnostic plan, execution plan⟩ pairs of the cost-based optimizer into a latent space ordered by their estimated latency. In each round, LSBO *explores* this space by sampling a batch of latent vectors from promising regions, and the VAE decoder *generates* an execution plan for each of them, conditioned on the platform-agnostic plan of the query. Valid candidates are executed, and their latencies guide the next round of sampling. Once a stopping condition is met, the plan with the lowest observed latency is used to retrain the classifier and the VAE.

A. Plan Generation

As the generative model, VARIBO uses a variational autoencoder (VAE). We choose a VAE for three reasons. First, conditional VAEs have been shown to perform well on structured output prediction [30], where each element of the output takes one of several labels, which directly corresponds to our setting, in which each operator is assigned a platform. Second,

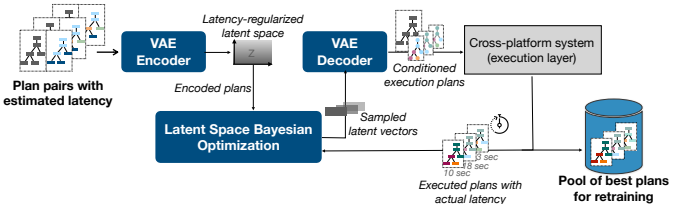


Fig. 3: Offline plan generation and exploration.

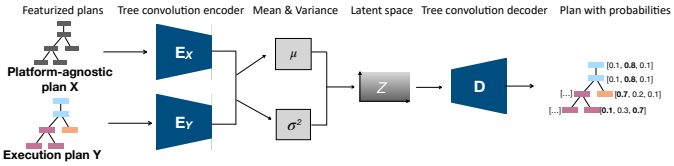


Fig. 4: VARIBO's generative model architecture.

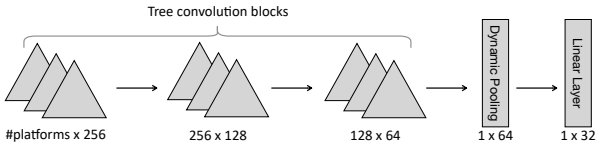


Fig. 5: Representation of the encoder layers. The decoder follows a similar architecture in reverse order.

a VAE encodes each input as a distribution over a continuous latent space [31], which enables both the generation of new execution plans and their systematic exploration in the latent space [32]. Third, a VAE is trained on plans alone, without any runtime labels, so that it can be bootstrapped from the plans of an existing cost-based optimizer.

1) Beyond simple VAE: While a standard VAE could already support both generation and exploration, we extend it to a *conditional attribute-regularized β-variational autoencoder* (CAR-β-VAE) to improve the quality of the generated plans and the efficiency of the exploration. We describe its components next.

β-weighting. The hyperparameter β controls the *disentanglement* of the latent space, i.e., the extent to which each latent dimension is sensitive to a single generative factor and largely unaffected by the others [33]. It governs a trade-off between reconstruction fidelity and latent capacity: values above 1 restrict the encoding capacity and encourage more disentangled representations. In our setting, this encourages distinct factors of a plan, such as its cardinalities and platform assignments, to occupy separate latent dimensions, which is intended to improve generalization to unseen plans.

Attribute regularization. To make the latent space directly useful for exploration, we regularize one of its dimensions with respect to latency [34], weighted by a hyperparameter γ. This makes latency an explicit dimension of the representation and orders the latent space accordingly, so that the exploration can steer the search toward regions associated with lower-latency execution plans. During bootstrapping, the regularization uses the latency estimates of the cost-based optimizer, which are replaced by measured latencies for the explored plans when the VAE is retrained. In our experiments (Section VI-F), exploration without this regularization takes about twice as long to find the best plan.

Conditioning. Finally, we condition the model on the platform-agnostic plan [30]. This ensures that every execution plan decoded from the latent space corresponds to the input query, i.e., it contains the same operators in the same structure and differs only in their platform assignments, regardless of where in the latent space the vector is sampled. Generation and exploration thus remain within the space of execution plans for the original query.

2) Model architecture: Figure 4 shows the architecture of the CAR-β-VAE. It takes as input a pair of a featurized platform-agnostic plan and a featurized execution plan (Section IV-A) and outputs a reconstructed execution plan where each operator is assigned the platform with the highest logit.

Encoder. The encoder takes as input a pair of featurized trees: the platform-agnostic plan X and the execution plan Y. It consists of two separate tree convolution networks, E_X and E_Y, one per input, with the same layer structure shown in Figure 5: an embedding layer, followed by two tree convolution layers, each with layer normalization, Mish activation, and dropout, and a dynamic pooling layer that turns the resulting tree into a fixed-size embedding. The two embeddings are concatenated and passed through linear layers that produce the mean μ and the log-variance $\log \sigma^2$ of the approximate posterior $q_\phi(z \mid X, Y)$. The latent vector is then computed as $z = \mu + \exp(0.5 \cdot \log \sigma^2) \cdot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, so that z is a sample from $\mathcal{N}(\mu, \sigma^2)$ [31]. The dimensionality of the latent space Z is much lower than that of the input, which acts as an information bottleneck [35]: it forces the model to retain only the essential aspects of a plan and to discard irrelevant details. Together with the attribute regularization, this places plans with similar structure and latency close to each other in the latent space. Besides being encoded, X also serves as the condition of the VAE: as both the encoder and the decoder receive it, the model learns latent representations specific to a given query.

Decoder. The decoder takes a latent vector $z \in Z$ together with the platform-agnostic plan X and outputs, for each operator, a vector of logits over the available platforms. To obtain an embedding of X, it reuses the network E_X of the encoder, which ensures consistent representations of X in encoding and decoding. This embedding is combined with z and passed through a mirrored architecture of linear and tree convolution layers, where the output dimension of the last tree convolution layer equals the number of available platforms. By conditioning on X, the decoder ensures that every generated execution plan corresponds to the original query, regardless of where in the latent space z is sampled.

3) Loss function: The CAR-β-VAE balances three objectives: reconstructing execution plans, learning a compact and

possibly disentangled latent representation, and ordering plans by latency in the latent space. Its loss function consists of one term per objective:

$$L_{\text{VARIBO}} = L_{\text{rec}} + \beta\, L_{\text{KL}} + \gamma\, L_{r,a} \tag{1}$$

where L_{rec} is the reconstruction loss, L_{KL} the KL divergence, and $L_{r,a}$ the attribute regularization loss, which we describe next. We use latency as the only attribute but the formulation extends to multiple attributes by adding a term per attribute.

Reconstruction loss. As the decoder outputs, for each operator, logits that a softmax turns into a probability distribution over platforms, we use the cross-entropy between the predicted and the actual platform assignments, summed over the operators of the plan:

$$L_{\text{rec}} = -\sum_{o \in Y} \sum_{p \in P} y_{o,p} \log \hat{y}_{o,p} \tag{2}$$

where $y_{o,p} \in \{0, 1\}$ indicates whether operator o is assigned to platform p in the execution plan Y, and $\hat{y}_{o,p}$ is the probability that the decoder assigns to p for o.

Kullback–Leibler divergence. The KL divergence term pulls the approximate posterior toward the prior $p(z) = \mathcal{N}(0, I)$ and thereby limits how much information about the input the latent representation can carry. For a Gaussian posterior, we have:

$$L_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \tag{3}$$

where d is the dimensionality of the latent space and μ_j and σ_j^2 are the mean and variance of the j-th dimension output by the encoder. As described above, $\beta > 1$ further restricts this capacity and encourages a disentangled representation.

Attribute regularization loss. Following the AR-VAE proposed in [34], the attribute regularization loss encodes an attribute in a designated latent dimension such that the attribute increases monotonically along that dimension. In our setting, the attribute is the latency of a plan. For a training batch, let D_r be the matrix of pairwise differences in the value of the designated latent dimension and D_a the matrix of pairwise differences in latency. The loss is:

$$L_{r,a} = \text{MAE}\left(\tanh(\delta D_r) - \text{sgn}(D_a)\right) \tag{4}$$

where MAE is the mean absolute error over all pairs and δ controls how sharply the loss separates plans whose latent values are close. The loss is minimized when, for every pair of plans, the order of their latent values matches the order of their latencies.

B. Plan Exploration

The CAR-β-VAE provides a continuous latent space in which plans are ordered by latency and every point can be decoded into an execution plan for the query. The remaining question is how to explore this space efficiently, i.e., how to find low-latency plans while executing as few candidates as possible. VARIBO uses latent space Bayesian optimization (LSBO), which is designed for expensive, noisy black-box objectives without gradients, such as plan latency: its surrogate model balances exploring uncertain regions with exploiting regions of known low latency. LSBO runs repeatedly in the background, and can thus cover queries and datasets added after the initial training.

Sampling the latent space. We formulate plan exploration for a platform-agnostic plan X as the following optimization problem over the latent space:

$$\min_{z \in Z} \ell\big(D(z, X)\big) \tag{5}$$

where $D(z, X)$ is the execution plan that the decoder produces for the latent vector z conditioned on X, and $\ell(\cdot)$ is its latency. LSBO fits a Gaussian process (GP) that models ℓ as a function of z. The seeds are the latent vectors that the encoder produces for X together with the execution plans of the cost-based optimizer, labeled with the optimizer's latency estimates in seconds so that LSBO can start without executing them. As the information bottleneck and the attribute regularization place plans with similar structure and latency close to each other in the latent space, plans that improve on a seed are likely to be found in its neighborhood. LSBO therefore uses Thompson sampling [36] on the GP to select a batch of latent vectors around the seeds, which the decoder turns into candidate execution plans for X.

Plan evaluation. Each valid candidate plan is executed on the cross-platform system to obtain its actual latency. Candidates that violate a platform constraint are discarded without execution and assigned a very high latency value. The latencies of a batch, including those of invalid candidates, are added to the observations of the GP to update the surrogate model, which guides the next batch toward regions of the latent space with lower latency.

Reducing exploration cost. Executing candidate plans is the most expensive step in the exploration phase. To reduce this cost, VARIBO bounds the execution time of each candidate by the lowest latency measured so far for the query. Executions that exceed this cutoff are terminated and labeled with a multiple of the cutoff, which steers the search away from similar regions without waiting for slow plans to finish. The cutoff is updated whenever a plan with lower latency is found.

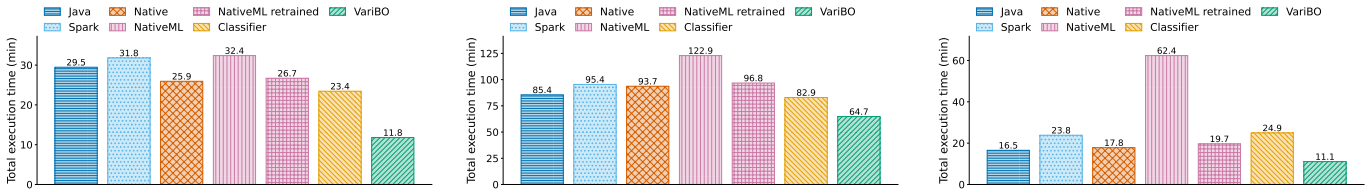
Stopping condition. Exploration continues until a stopping condition is met, such as a time budget, a maximum number of executed candidates, or a target latency improvement over the seed plans. The plan with the lowest observed latency for each explored query is then added to the training data pool, on which the classifier and the CAR-β-VAE are incrementally updated rather than retrained from scratch, amortizing the cost of retraining over time.

VI. EXPERIMENTAL EVALUATION

We implement VARIBO in Apache Wayang[2], an open-source system that offers cross-platform execution and comes with a cost-based cross-platform optimizer. Our implementation and all hyperparameters are publicly available.[3] We answer five questions. (i) How efficient are the plans of VARIBO

[2]https://wayang.apache.org/

[3]https://github.com/juripetersen/VariBO



(a) JOB-C (b) STATS (c) TPC-H

Fig. 6: Total workload runtime (sum of all query latencies, in minutes) of each optimizer per benchmark. Lower is better.

compared to those of the baselines, for both explored and unexplored queries (Section VI-B)? (ii) Which platforms does VARIBO choose compared to the baselines (Section VI-C)? (iii) How efficient is its online optimization (Section VI-D)? (iv) What makes its plan exploration efficient (Section VI-F)? (v) What does the offline phase cost, and when is this cost amortized (Section VI-G)?

A. Setup

Hardware and Software. All experiments run on identical UCloud[4] nodes, each with an AMD EPYC 9535, 16 vCPUs, and 48 GB of RAM. We deploy Apache Wayang 0.7.1 with its Java platform on one node, PostgreSQL 17.2 on another, and Apache Spark 3.5.7 on three nodes with one master and two workers.

Benchmarks. To the best of our knowledge, there is no dedicated benchmark for cross-platform query optimization. We therefore use three benchmarks that are widely used for relational query optimization and parse each SQL query into a platform-agnostic plan with Wayang's SQL API, which is based on Apache Calcite [37]. We use JOB-C [38][5] with 30 test queries, of which VARIBO explores 15, and STATS [39] with 146 test queries, of which VARIBO explores 30. Their test queries are the original benchmark queries. For STATS, the training and validation queries are those generated from the benchmark queries in [39]. For JOB-C, we use the synthetic JOB queries of [17], which were generated over the IMDb schema for the JOB workload in general rather than from the JOB-C queries, so the JOB-C test queries are structurally more complex than the queries VARIBO is trained on. To emulate a heterogeneous environment, we store some of the tables of both benchmarks in PostgreSQL and the others as CSV files. The exact split can be found in our repository. We further use TPC-H [40] at scale factor 3 with all tables stored as CSV files and generate all of its queries with DataFarm [20] from templates 1, 3, 5, 6, 10, 12, 14, and 19. VARIBO explores all 30 TPC-H test queries, which represents a recurring workload drawn from a fixed set of templates. For each benchmark, we split the 1,000 queries into 900 training and 100 validation queries. On JOB-C and STATS, we select the explored queries uniformly at random with a fixed seed. The remaining *unexplored* queries share schema and data (and, on STATS and TPC-H, also their templates) with the training queries, and VARIBO optimizes them with the retrained classifier alone.

Baselines. An optimal-plan oracle is infeasible: a plan with 20 operators and 3 platforms has up to $3^{20} \approx 3.5 \times 10^9$ execution plans, each of which would have to be executed. We therefore compare VARIBO against single-engine execution and other cross-platform optimizers. As single-engine baselines, we consider JAVA and SPARK, which execute all operators on Wayang's single-node Java platform or on Spark, respectively (on JOB-C and STATS, PostgreSQL only scans the tables stored in it). As cross-platform optimizers, we consider (i) NATIVE, Wayang's cost-based optimizer with hand-written cost formulas, (ii) NATIVEML, a learned cost model approach inspired by [23], which replaces these formulas with a learned cost model but keeps plan enumeration, (iii) NATIVEML RETRAINED, which we retrain on the measured latencies of the plans that VARIBO executes during exploration, so that both use the same plan executions, and (iv) CLASSIFIER, an ablation of VARIBO without offline exploration. As is common practice for learned optimizers [15]–[17], we use NATIVE with its default cost model. In a cross-platform setting, calibration requires tedious tuning of the cost formulas of every platform and every operator (including data movement costs) to the hardware and data at hand [23], which is exactly what learned optimizers aim to avoid. We implement NATIVEML with the same tree-convolutional architecture and plan encoding as VARIBO and bootstrap it from the estimates of NATIVE (Section VI-G). The enumeration of NATIVE, NATIVEML, and NATIVEML RETRAINED uses top-k pruning [12], which keeps only the $k = 100$ candidates with the lowest estimated cost at each step. Without pruning, enumerating a single JOB-C query does not finish within one hour (Section VI-D). All optimizers run through Wayang's execution layer and operator implementations, so that runtime differences reflect the choice of platforms and not the execution implementations.

Configuration. We report the mean of five runs for all final execution plans, while candidate plans executed during exploration run once. We evaluate VARIBO after one round of exploration and retraining. Exploration of a query stops after 1,000 generated candidates, or earlier if the best runtime has not improved during the last quarter of this budget. Invalid candidates are discarded without execution but count toward the budget, and slow candidates are terminated

[4]https://docs.cloud.sdu.dk/

[5]https://github.com/DataManagementLab/JOB-Complex

early (Section V-B). We use a candidate budget instead of a time budget so that short- and long-running queries get a comparable chance to be explored. We found empirically that 1,000 candidates balance plan quality and exploration time. We tune the hyperparameters of VARIBO, NATIVEML, and NATIVEML RETRAINED with Bayesian optimization, running 50 trials per model and selecting the configuration with the lowest validation loss. We exclude the loss weights β and γ and the margin δ of the CAR-β-VAE (Section V-A) from this search and set them manually, because each of them controls a trade-off and not a quantity to minimize [34]. With $\beta = 1.5$, the model favors reconstruction while still encouraging a disentangled latent space. With $\gamma = 4$, it emphasizes the discriminative power of the latency dimension, and $\delta = 2$ enforces a clear latency ordering without destabilizing training. In preliminary experiments, deviating from these values affected model performance in the ways reported in the original papers of these parameters [34], [41].

B. Effectiveness on Query Performance

We first evaluate how efficient the plans of VARIBO are compared to those of the baselines, both in total workload runtime and per query, and distinguish between recurring and new queries. *In summary,* VARIBO *is the fastest optimizer on all three benchmarks, with a speedup over* NATIVE *of* $2.20\times$ *on JOB-C,* $1.45\times$ *on STATS, and* $1.60\times$ *on TPC-H. Its gains stem from mixed-platform plans found through exploration, which speed up long-running queries without noticeably slowing down short ones. In addition, retraining carries the gains over to unexplored queries as long as the explored queries are representative of the workload.*

Workload performance. Figure 6 shows that VARIBO achieves the lowest total workload runtime on all three benchmarks. Compared to NATIVE, it reduces the workload runtime from 25.9 to 11.8 min on JOB-C ($2.20\times$), from 93.7 to 64.7 min on STATS ($1.45\times$), and from 17.8 to 11.1 min on TPC-H ($1.60\times$). It also outperforms the best single platform, JAVA, which requires 29.5, 85.4, and 16.5 min, i.e., up to $2.50\times$ longer than VARIBO. Notably, JAVA is faster than NATIVE on STATS and TPC-H, yet VARIBO outperforms both. The plans of VARIBO are faster because they shift part of the computation from the single-node Java platform to the Spark cluster. Although this increases the number of data movements per query, the parallel execution on Spark more than compensates for them. NATIVE misses these plans because its cost formulas fail to capture when Spark pays off for individual operators within a mixed plan. The learned baselines do not close this gap. NATIVEML RETRAINED requires 26.7, 96.8, and 19.7 min, i.e., up to $2.26\times$ longer than VARIBO, although it is trained on the same plan executions, as its learned model does not rank the faster mixed plans among its top-100 candidates. NATIVEML is even slower than NATIVE on all benchmarks, requiring 32.4, 122.9, and 62.4 min, i.e., up to $5.62\times$ longer than VARIBO. As it enumerates the same candidates as NATIVE and is trained on the estimates of NATIVE, a faithful cost model would select the same plans; its slower plans thus show that the learned cost model fails to reproduce even the estimates it is trained on. Finally, CLASSIFIER requires 23.4, 82.9, and 24.9 min, i.e., up to $2.24\times$ longer than VARIBO: as it only imitates the plans of NATIVE, it rarely uses Spark where VARIBO does.

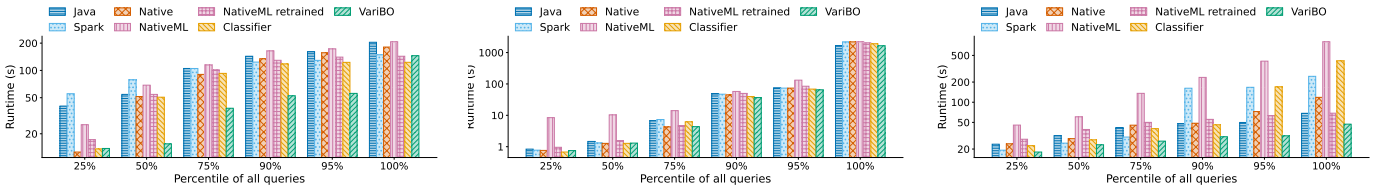
TABLE I: Speedup over NATIVE in total runtime of recurring queries (Explored) and of new queries (Unexplored). Values above $1.0\times$ indicate a speedup over NATIVE. TPC-H has no unexplored queries, since VARIBO explores all of them.

Query subset	Optimizer	JOB-C	STATS	TPC-H
Explored	NATIVEML	$0.82\times$	$0.84\times$	$0.29\times$
	NATIVEML RETR.	$0.90\times$	$1.11\times$	$0.90\times$
	CLASSIFIER	$0.99\times$	$1.10\times$	$0.71\times$
	VARIBO	$\mathbf{1.83}\times$	$\mathbf{2.03}\times$	$\mathbf{1.60}\times$
Unexplored	NATIVEML	$0.79\times$	$0.70\times$	–
	NATIVEML RETR.	$1.00\times$	$0.87\times$	–
	CLASSIFIER	$1.16\times$	$1.16\times$	–
	VARIBO	$\mathbf{2.39}\times$	$\mathbf{1.16}\times$	–

Per-query performance. To determine whether the workload gains stem from many queries or from a few long-running ones, Figure 7 shows the 25th to 100th percentiles of the per-query runtimes on a logarithmic scale. On JOB-C and TPC-H, the gains of VARIBO are spread across the distribution: on JOB-C, it has the lowest runtime from the median to the 95th percentile (e.g., about 15 vs. 50 s at the median compared to NATIVE), and on TPC-H at every percentile, with its slowest query completing in under 50 instead of about 120 s. On STATS, the gains are concentrated in the tail: VARIBO matches NATIVE up to the 75th percentile, where most queries complete in about 1 s and leave little room for improvement, but has the lowest runtime at the 90th and 95th percentiles, which dominate the workload runtime. Exploration also removes the long tail of CLASSIFIER on TPC-H, which stems from a few queries for which it selects plans of several hundred seconds.

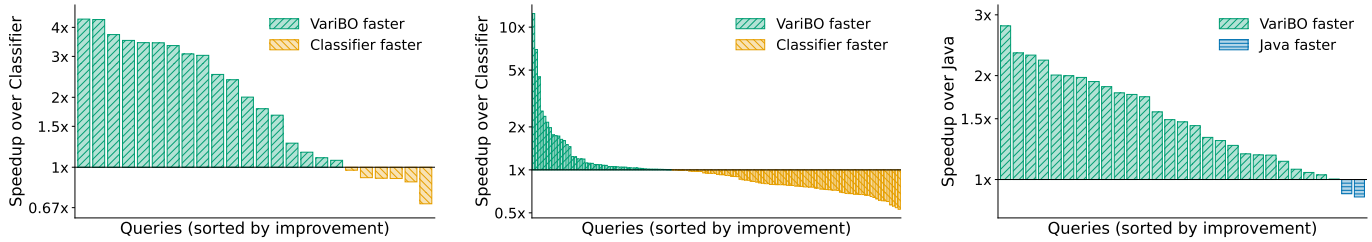
We additionally study the runtime of each query for VARIBO and NATIVE (Figure 9 shows the per-query comparisons). VARIBO is within 10% of NATIVE or faster on 24 of 30 queries on JOB-C, 117 of 146 on STATS, and 29 of 30 on TPC-H. The remaining queries exhibit only minor regressions: together, they add 11.7, 19.4, and 3.5 s to the workload runtimes, i.e., about 1% of the 14.1, 29.0, and 6.7 min that VARIBO saves, so the improvements clearly outweigh them. Figure 8 additionally summarizes the per-query comparison of VARIBO with the next best optimizer, i.e., CLASSIFIER on JOB-C and STATS and JAVA on TPC-H. VARIBO is faster on 18 of 30 queries on JOB-C, by up to $4.34\times$, and at most $1.44\times$ slower. It is faster on 27 of 30 queries on TPC-H, by up to $2.79\times$, while the remaining queries are at most $1.12\times$ slower. On STATS, VARIBO is faster on 50 of the 146 queries, by up to $12.44\times$, and slower in 82 by less than $1.89\times$. On the remaining queries, both optimizers select the same plan.

Recurring and new queries. Table I reports the speedup over NATIVE of VARIBO, NATIVEML, NATIVEML RETRAINED,



(a) JOB-C (b) STATS (c) TPC-H

Fig. 7: Per-query runtime percentiles of each optimizer on (a) JOB-C, (b) STATS, and (c) TPC-H (log scale, lower is better). VARIBO is faster from the median to the 95th percentile on JOB-C, at every percentile on TPC-H, and in the tail on STATS.



(a) JOB-C (b) STATS (c) TPC-H

Fig. 8: Per-query speedup of VARIBO over second-best optimizer of each benchmark in total runtime (higher is better). VARIBO is faster on most queries of JOB-C and TPC-H. On STATS, its gains concentrate on fewer queries with large speedups.

and CLASSIFIER separately for the recurring queries that VARIBO explored (15 of 30 on JOB-C, 30 of 146 on STATS, and all queries on TPC-H) and for the new, unexplored ones. On the explored queries, VARIBO achieves a speedup over NATIVE of 1.83×, 2.03×, and 1.60×, whereas NATIVEML RETRAINED, although trained on the measured runtimes of exactly these queries, achieves only 0.90×, 1.11×, and 0.90×, as the faster plans are not among its candidates after pruning. On the unexplored JOB-C queries, VARIBO achieves 2.39×, compared to 1.16× for CLASSIFIER: retraining on the best plans of the explored queries carries the improvements over to the remaining queries. On STATS, in contrast, VARIBO achieves 1.16× on the unexplored queries, the same as CLASSIFIER; the explored queries, which make up around a fifth of the workload, do not cover the remaining queries well enough for the improvements to transfer. Retraining can also lose an improvement on an unexplored query: for the slowest JOB-C query, CLASSIFIER selects a plan that runs in 101 s, whereas VARIBO returns to the plan of NATIVE, which runs in 145 s.

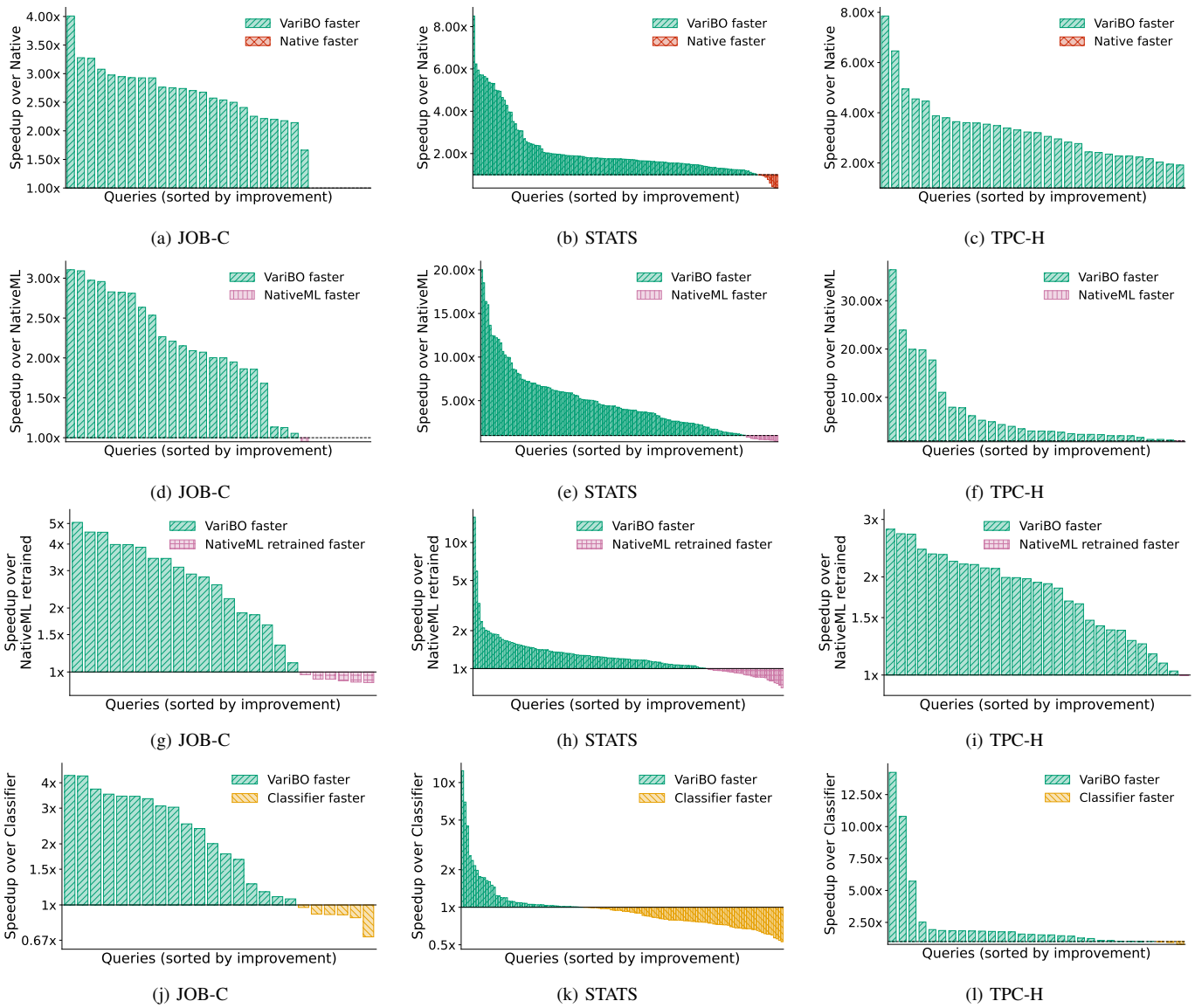
Transfer to new datasets. Beyond new queries on the same dataset, we also investigate VARIBO on entirely new datasets, with different schemas, data, and queries. To this end, we train, explore, and retrain VARIBO only on JOB-C and apply it to STATS and TPC-H without any further training or exploration. In this setting, VARIBO performs on par with NATIVE on STATS (1.02×) and slightly better on TPC-H (1.13×) in total workload runtime, and it is faster than NATIVE on more than half of the queries of both datasets. The transferred model thus provides a safe starting point for a new dataset, while realizing the gains observed in the in-distribution setting requires exploring part of its queries. Due to space limitations and as transfer to new datasets is not the main focus of this paper, we provide a detailed analysis of this experiment in our repository [42].

TABLE II: Platform assignments for the test queries: share of operators per platform and total number of data movements.

		Operators (%)			# Data
Benchmark	**Optimizer**	Java	Spark	PostgreSQL	**movements**
JOB-C	NATIVE	59.2	2.2	38.6	181
	NATIVEML	61.6	2.8	35.6	193
	NATIVEML RETR.	60.6	2.7	36.7	190
	CLASSIFIER	60.0	2.3	37.8	187
	VARIBO	54.0	8.2	37.8	242
STATS	NATIVE	83.0	0.0	17.0	248
	NATIVEML	75.2	11.2	13.6	665
	NATIVEML RETR.	86.8	0.0	13.2	263
	CLASSIFIER	89.6	0.3	10.1	284
	VARIBO	61.0	22.0	17.0	603
TPC-H	NATIVE	98.0	2.0	-	5
	NATIVEML	90.0	10.0	-	60
	NATIVEML RETR.	99.3	0.7	-	3
	CLASSIFIER	86.5	13.5	-	18
	VARIBO	24.4	75.6	-	35

C. Analysis of Platform Assignments

To understand where the gains of VARIBO originate, we analyze which platforms the plans of VARIBO and the cross-platform baselines use. *In summary,* VARIBO *pushes the operators that PostgreSQL supports down to PostgreSQL, like* NATIVE*, and additionally moves part of the remaining computation from Java to Spark where the parallel execution pays off.* NATIVEML *pushes fewer operators down to*



(a) JOB-C (b) STATS (c) TPC-H

(d) JOB-C (e) STATS (f) TPC-H

(g) JOB-C (h) STATS (i) TPC-H

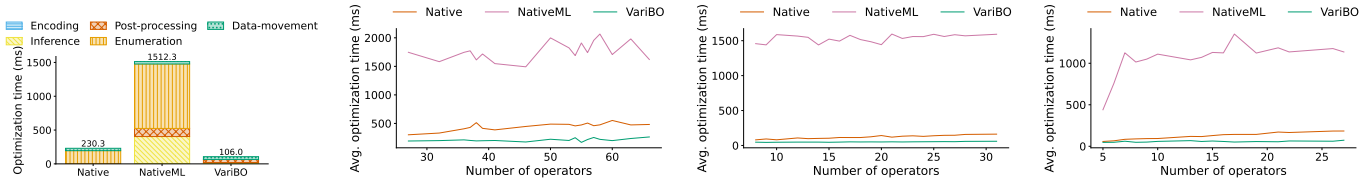
(j) JOB-C (k) STATS (l) TPC-H

Fig. 9: Per-query improvement of VARIBO over the four baselines (higher is better). The top row compares against NATIVE, the upper middle row against NATIVEML, the lower middle row against NATIVEML RETRAINED and the bottom row against CLASSIFIER. Columns correspond to the JOB-C, STATS, and TPC-H benchmarks. VARIBO finds better plans for the large majority of queries.

PostgreSQL, which increases its data movements, while the remaining baselines rarely use Spark on JOB-C and STATS.

Table II reports the percentage of operators that each platform executes in the test queries for all cross-platform optimizers. It also reports the total number of data movements over all test queries, i.e., the number of edges whose operators run on different platforms. NATIVE pushes the filters and projections on the tables stored in PostgreSQL down to PostgreSQL (38.6% and 17% of the operators on JOB-C and STATS) and executes almost all remaining operators on Java: it assigns only 2.2%, 0%, and 2% of the operators to Spark on JOB-C, STATS, and TPC-H. This preference for Java is consistent with the single-platform baselines. On TPC-H, where all data is CSV files, PostgreSQL is not used.

NATIVEML deviates from this pattern in a way that hurts performance. On JOB-C and STATS, it assigns fewer operators to PostgreSQL than NATIVE (35.6% instead of 38.6%, and 13.6% instead of 17%), i.e., it does not push down all operators that PostgreSQL could execute, so that unfiltered data has to be moved out of PostgreSQL. This increases the number of data movements from 181 to 193 on JOB-C and from 248 to 665 on STATS, and makes NATIVEML the slowest cross-platform optimizer on both benchmarks (32.4 and 122.9 min). On TPC-H, it assigns 10% of the operators to Spark with 60 data movements, 12× as many as NATIVE, resulting in the slowest workload runtime of all optimizers (62.4 min). NATIVEML



(a) Average (b) JOB-C (c) STATS (d) TPC-H

Fig. 10: Online optimization time of each optimizer: (a) Average optimization time and breakdown. (b)–(d) Optimization time scaling with plan size. VARIBO scales better than enumeration-based optimizers.

RETRAINED and CLASSIFIER stay close to NATIVE: they assign at most 2.7% of the operators to Spark on JOB-C and STATS. On STATS, where they also push fewer operators to PostgreSQL (13.2% and 10.1%), they keep the remaining operators on Java, so their data movements increase only moderately (263 and 284). NATIVEML RETRAINED, although trained on the measured runtimes of VARIBO's plans, assigns only 0.7% of the operators to Spark on TPC-H, as the mixed plans are not in the candidates retained by top-k pruning.

VARIBO, in contrast, first pushes down to PostgreSQL what PostgreSQL can execute: its share of PostgreSQL operators matches NATIVE on STATS (17%) and nearly on JOB-C (37.8%). It then moves the remaining computation to Spark where the parallel execution pays off and keeps light-weight operators on Java, assigning 8.2%, 22%, and 75.6% of the operators to Spark on JOB-C, STATS, and TPC-H. On TPC-H, exploration raises this share from 13.5% for CLASSIFIER to 75.6%. Combining Java and Spark within a plan increases the number of data movements to 242, 603, and 35, respectively. On STATS, this is almost as many as NATIVEML, yet VARIBO is the fastest optimizer: its data movements transfer data that PostgreSQL has already filtered, and the parallel execution on Spark more than compensates for them. Notably, as SPARK alone is slower than JAVA on every benchmark, no single platform for the whole workload achieves these gains: VARIBO instead chooses platforms per query, executing some queries entirely on one platform and combining Java and Spark within others, where the parallel execution outweighs the additional data movements.

D. Query Optimization Efficiency

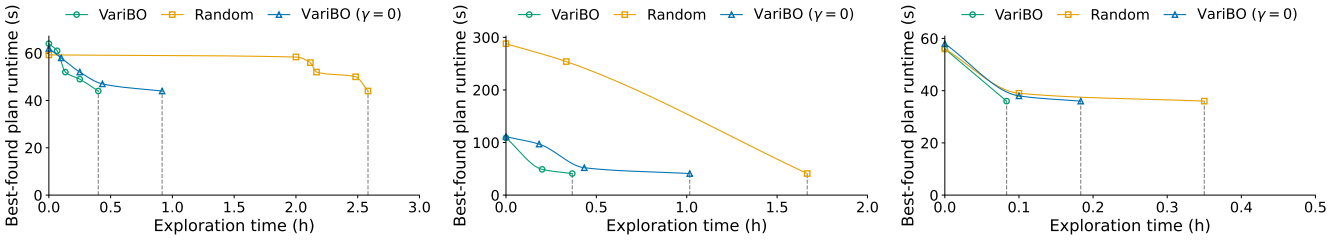
In summary, VARIBO *optimizes queries about* 2× *faster than* NATIVE *and* 14× *faster than* NATIVEML *and* NATIVEML RETRAINED*, since it predicts all platform assignments in a single pass rather than enumerating candidate plans. Moreover, its advantage over* NATIVE *grows with plan size on STATS and TPC-H.*

Average optimization time. Figure 10a reports the average optimization time across all benchmarks, decomposed into five steps. *Plan enumeration*, which uses top-k pruning [12], is performed only by NATIVE and NATIVEML. *Encoding* featurizes a plan, and *inference* executes the model using ONNX Runtime; both steps apply only to the learned optimizers. *Post-processing* converts the model outputs into Wayang's data structures, derives the platform assignments from them, and applies the validation rules, and *data movement* inserts the operators that move data at platform switches. We omit NATIVEML RETRAINED and CLASSIFIER, as they follow the same optimization path as NATIVEML and VARIBO, respectively. NATIVE spends 195.3 ms on enumeration and 35 ms on data-movement. NATIVEML additionally performs inference and a pairwise comparison for every enumerated candidate before pruning, which increases its average optimization time to 1,512 ms, 14× more than VARIBO. VARIBO requires only 106 ms, as it performs a single inference.

Scaling with plan size. Figures 10b–10d show the optimization time of VARIBO, NATIVE, and NATIVEML as a function of the number of operators in a plan. The optimization time of the enumeration-based optimizers grows with plan size: on JOB-C, NATIVE requires about 300 ms for the smallest plans (27 operators) and about 480 ms for the largest ones (66 operators), and on STATS and TPC-H its optimization time roughly doubles and triples, respectively, from the smallest to the largest plans. NATIVEML remains above NATIVE for all plan sizes, at about 1.5 to 2 s on JOB-C, as it runs inference for every candidate generated during enumeration before pruning them. The optimization time of VARIBO, in contrast, grows only linearly with the number of operators, as it performs a single inference over the plan: on JOB-C, it increases only from about 190 to 260 ms. This gap between VARIBO and the enumeration-based optimizers matters in the cross-platform setting, where the plan space grows exponentially with the number of operators and platforms. Moreover, it is conservative: NATIVE and NATIVEML already use top-100 pruning. Without it, enumerating the plans of a single JOB-C query took longer than executing the entire JOB-C workload (over one hour).

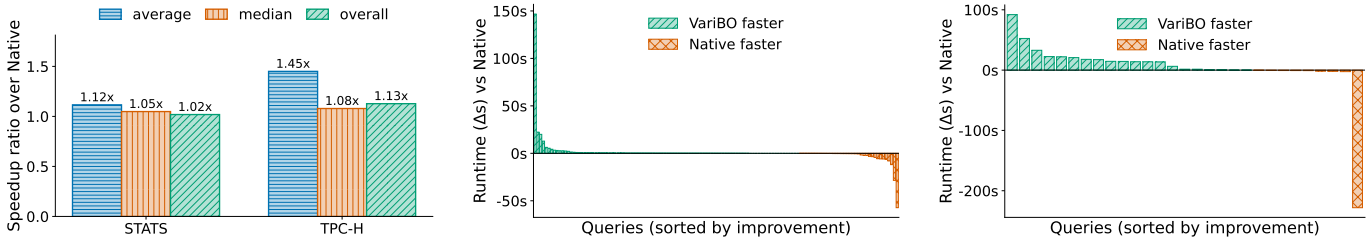
E. Out-of-distribution Performance

We now evaluate whether VARIBO transfers its optimization knowledge to unseen workloads with different schemas, data, and queries. Without any training or exploration on the target workloads, VARIBO is on par with NATIVE on STATS (1.02×) and faster on TPC-H (1.13×) in total workload runtime, and it is faster on the majority of the queries of both. The transfer is, however, considerably weaker than in-distribution optimization, and a few large regressions remain.



(a) JOB-C Q14 (b) STATS Q113925678 (c) TPC-H Q12

Fig. 11: Runtime of the best plan found so far over exploration time. All three converge to a plan with the same runtime, but VARIBO finds it considerably sooner. Dashed lines mark the time at which each variant finds its best plan.



(a) Aggregate speedup over NATIVE (b) STATS: per-query runtime vs. NATIVE (c) TPC-H: per-query runtime vs. NATIVE

Fig. 12: Out-of-distribution performance. VARIBO is trained and explored on JOB-C and then applied to STATS and TPC-H without further training or exploration. (a) reports the overall, average, and median speedup over NATIVE per dataset, and (b) and (c) show the absolute runtime difference to NATIVE per query. VARIBO matches the performance of NATIVE at the workload level on both datasets.

We train, explore, and retrain VARIBO only on JOB-C and then apply it to STATS and TPC-H without further training, exploration, or retraining. We choose JOB-C as the source workload because its complex multi-join queries are generally long-running and provide a rich signal for optimization. STATS is a moderate shift: like JOB-C, it reads from PostgreSQL and CSV files, but most of its queries are short-running, and only a few long-running queries dominate its total runtime. TPC-H is the largest shift: it reads only CSV files, and its queries are neither as long-running nor as join-heavy as those of JOB-C. Figure 12a reports three speedups over NATIVE: the *overall* speedup in total workload runtime, as in Section VI-B, and the *average* and *median* of the per-query speedups, which weigh every query equally regardless of its runtime. Figures 12b and 12c show the absolute runtime difference to NATIVE per query, sorted by improvement.

Workload performance. On both target workloads, all three speedups are above $1.0\times$. On STATS, the overall speedup is $1.02\times$, the average $1.12\times$, and the median $1.05\times$. On TPC-H, the overall speedup is $1.13\times$, the average $1.45\times$, and the median $1.08\times$. Since the median is above $1.0\times$, VARIBO is faster than NATIVE on more than half of the queries of both workloads. The average is higher than the overall speedup on both workloads, so the relative gains are larger on the queries that contribute little to the total runtime. To put these numbers into perspective, we compare them to the in-distribution results of Section VI-B. There, VARIBO reaches $1.45\times$ on STATS and $1.60\times$ on TPC-H, so transfer alone recovers only a small part of the gains of in-distribution exploration. The transferred model is nevertheless competitive with the in-distribution baselines. On TPC-H, it is faster than the best single platform (JAVA, $1.08\times$) and than CLASSIFIER ($0.71\times$), which is bootstrapped on TPC-H itself. The knowledge gained from exploring JOB-C thus helps more on TPC-H than imitating NATIVE on TPC-H. On STATS, it stays below JAVA ($1.10\times$) and CLASSIFIER ($1.13\times$).

Per-query performance. The per-query differences show why the overall speedups are moderate. On STATS (Figure 12b), the runtime of most queries changes by less than a second. A few queries improve substantially, the largest by about 150 s. As in Section VI-B, we count the queries for which VARIBO is more than 10% slower than NATIVE. On STATS, these are 23 of 146 queries, which together add 104 s to the workload runtime, the largest single regression being about 55 s. Since VARIBO saves only about 103 s on STATS in total, the regressions cancel about half of its gains. On TPC-H (Figure 12c), about half of the queries improve by 10 to 90 s, and only 3 of 30 queries regress. These three, however, add 233 s to the workload runtime, almost all of it from a single query that regresses by about 230 s. If VARIBO had kept the plans of NATIVE for the regressing queries, its overall speedup would be $1.04\times$ on STATS and $1.49\times$ on TPC-H, close to the in-distribution $1.60\times$ on TPC-H. The large regression on TPC-H occurs on query 5 of our test set. For this query, VARIBO

alternates between Spark and Java within the plan, so that intermediate results are moved back and forth between the two platforms. On JOB-C, mixing Spark and Java pays off because the parallel execution of large joins on Spark outweighs the cost of the data movements (Section VI-C). Applied to TPC-H without exploration, VARIBO transfers this pattern to a query where the data movements outweigh the benefit. Transfer thus finds better plans for most queries, but a few large regressions limit its overall benefit.

Summary. VARIBO transfers to unseen workloads without any runtime labels on them: it is faster than or on par with NATIVE at the workload level, faster on most queries, and on TPC-H it even outperforms the best single platform and CLASSIFIER bootstrapped on TPC-H itself. Transfer alone, however, leaves most of the in-distribution gains unrealized and does not prevent individual large regressions. These regressions are what the offline phase of VARIBO addresses: exploring a part of the new workload and retraining on it would adapt VARIBO to this workload, just as exploring half of JOB-C improved its unexplored queries in the in-distribution setting (Section VI-B).

F. Plan Exploration Efficiency

We now examine whether the improvements of VARIBO stem from its search strategy rather than solely from the learned latent space, and how exploration affects the validity of the predicted plans. *In summary, on the analyzed queries,* VARIBO *finds its best plan 4–6.5× faster than random sampling from the same latent space, and 2–2.5× faster than with a latent space that is not ordered by latency. Moreover, retraining on executed plans reduces the number of invalid platform assignments.*

Search efficiency. We compare VARIBO against two variants. RANDOM samples candidates from the same latent space without the guidance of the LSBO surrogate model (Section V), isolating the effect of Bayesian optimization. VARIBO ($\gamma = 0$), on the other hand, retains LSBO but trains the VAE without attribute regularization, so that the latent space is no longer ordered by latency. This variant isolates the effect of the regularized latent space. All variants operate under the same budget of 1,000 candidates, with the same stopping criteria and early termination. For one explored query per benchmark, Figure 11 shows the runtime of the best plan found so far (y-axis) as a function of exploration time (x-axis). On all three queries, the variants eventually converge to a plan with the same runtime, but they differ substantially in the time required to find it. On the JOB-C query (Figure 11a), VARIBO finds its best plan after about 0.4 h, compared to about 0.9 h for VARIBO ($\gamma = 0$) and 2.6 h for RANDOM. The STATS query (Figure 11b) exhibits the same ordering (0.4 h, 1.0 h, and 1.7 h), with RANDOM also starting from considerably slower plans. On the TPC-H query (Figure 11c), VARIBO requires only about 5 minutes, while VARIBO ($\gamma = 0$) takes about 2× as long and RANDOM about 4× as long. Since all variants sample from the same learned latent space, they all find good plans eventually; VARIBO, however, finds them with fewer executions, as its surrogate model guides the search toward promising regions, and the latency-ordered latent space makes this guidance more effective. As executing candidate plans dominates the offline cost, this directly reduces the cost of exploration (Section VI-G).

TABLE III: Average number of validation rule activations per query on the classifier's predicted plans, before and after one round of exploration and retraining.

Benchmark	Before	After	Reduction (%)
JOB-C	7.9	6.3	19.5
STATS	3.1	2.7	13
TPC-H	0.0	0.0	–

TABLE IV: Offline exploration cost and break-even point relative to NATIVE: number of workload (and query) executions after which the saved runtime exceeds the exploration time.

	JOB-C	STATS	TPC-H
Queries (total / explored)	30 / 15	146 / 30	30 / 30
Exploration time (h)	38.9	20.0	42.4
Workload runtime, NATIVE (min)	25.9	93.7	17.8
Workload runtime, VARIBO (min)	11.8	64.7	11.1
Time saved per execution (min)	14.1	29.0	6.7
Break-even (workload executions)	**166**	**42**	**380**
Break-even (query executions)	4,980	6,132	11,400

Plan validity. For Wayang, VARIBO requires four validation rules (Section IV). Since VARIBO retrains only on plans that execute successfully, exploration shifts the classifier toward valid platform assignments: after exploration, these rules are activated on average 19.5% less often per query on JOB-C and 13.0% less often on STATS. The remaining activations are inexpensive, as each selects the next most probable supported platform. They are part of the post-processing step and thus included in the optimization time of VARIBO (Section VI-D). On TPC-H, the rules are never activated, as both JAVA and SPARK support every operator in its plans. We report the number of rule activations per query in our repository [42].

G. Offline Cost and Amortization

We finally quantify the offline cost of VARIBO and when it pays off. *In summary,* VARIBO *collects its initial training data in* 166 *min per benchmark without executing any plan, whereas runtime labels for a learned cost model would require an estimated* 41 *to* 60 *days of plan executions. Its exploration takes* 20 *to* 42 *h and pays off after* 42 *to* 380 *executions of the respective workload, a volume that production analytical workloads reach quickly.*

Training data collection. VARIBO collects its initial training data without executing any plan, as it relies on the plans of NATIVE and their estimated runtimes. For the 1,000 training and validation queries of each benchmark, this collection takes 166 minutes as it collects 100 candidate plans per training query. In contrast, collecting runtime labels for a learned cost model requires executing the 100 plans that top-k

pruning retains for each training query, amounting to 100,000 plan executions per benchmark. Assuming the mean query runtime of NATIVE on each test workload, this would take about 41 to 60 days per benchmark, and likely more, as most candidate plans are slower than the plan selected by NATIVE. To keep our experiments feasible, and so that both models start from the same information, we bootstrap NATIVEML from the estimates of NATIVE as well, and consider NATIVEML RETRAINED, which reuses the runtimes measured during the exploration of VARIBO, as a separate baseline.

Exploration cost and break-even. Exploration constitutes the main offline cost of VARIBO, as it executes candidate plans to measure their runtimes. Unlike eager labeling, however, it executes plans *lazily*, only where the search finds them promising. Table IV relates the resulting offline time to the time that VARIBO saves per workload execution compared to NATIVE. Exploration takes 38.9 h on JOB-C, 20.0 h on STATS, and 42.4 h on TPC-H, and pays off after 166, 42, and 380 executions of the respective workload. STATS breaks even fastest, as VARIBO explores only 30 of its queries, which keeps the offline time low, while saving 29.0 min per workload execution on its long-running queries. TPC-H takes longest, as VARIBO explores all of its queries but saves only 6.7 min per execution on an already short-running workload. This amortization relies on queries being executed repeatedly, which is typical for production analytical workloads [24]. Moreover, query volumes in practice are high: in the Snowflake Data Cloud, BI tools alone issued 667 million queries within two weeks [43]. At such volumes, the 5,000 to 11,000 query executions required to break even are reached quickly. Finally, the offline cost does not block query processing, as VARIBO optimizes queries with its current classifier while exploration runs in the background.

VII. RELATED WORK

We discuss related work in three areas: systems that process queries across multiple engines, cross-platform query optimization, and learned query optimization.

Multi-engine query processing. Several classes of systems involve multiple engines in answering a query, but differ from cross-platform systems in the role the engines play. Federated databases answer queries over data that stays in its sources: they push operations down to the sources and combine the results in a mediator [6], [44] or let the sources exchange data and compute the result among themselves [7], [8]. Polystores [9] integrate engines with different data models and optimize queries mainly within groups of engines that share a data model, with explicit casts to move data between groups. Query routing systems [10], [11] send each query as a whole to the engine expected to run it fastest. In contrast, the platforms of a cross-platform system are *processing engines* that can execute many of the same operations on the same data, so that a single query can be split across them, with each operator running on the engine best suited for it. This exposes opportunities that whole-query routing misses, but makes the search space grow exponentially with the number of operators.

Cross-platform query optimization. Cross-platform optimizers select a platform per operator using cost models [4], [5], [12], which must be calibrated for each platform and dataset. A recent study [1] confirms that the choice of where to split a query matters. The work in [23] replaces the cost model of a cross-platform optimizer with a learned model that estimates plan runtime and uses it to prune plans during enumeration. Its quality therefore depends on the accuracy of the runtime estimates over all enumerated plans. VARIBO instead predicts a platform per operator in a single pass, without enumeration, and learns from executed plans that its exploration selects.

Learned query optimization for databases. ML has been widely applied to enhance or replace the cost-based optimizer of a DBMS [2], [13], [15]–[18], [22], [45]. As accurate cost or latency prediction is difficult [13], [15], [17], [46], later works rank plans instead of predicting their cost [13], [17], [22], or steer the existing optimizer with hints [15], [17], [18]. Like VARIBO, Neo [16] starts from an expert optimizer's plans, but learns a value model from executed plans and searches over plans at query time. All of these operate within a single DBMS, and the hint-based ones rely on hint sets that cross-platform systems do not offer. Closest to VARIBO are generative approaches. BayesQO [47] pairs a VAE with Bayesian optimization in its latent space, a technique originally developed for discrete structures such as molecules [32], [48], to find better plans for repeated queries offline, and GenJoin [49] uses a conditional generative model to produce subplan hints. Both target a single database: BayesQO optimizes each query separately and falls back to the existing optimizer for unseen queries, and GenJoin steers the existing optimizer. In contrast, VARIBO uses exploration to produce training data for a classifier that serves any query online, including unseen ones for the cross-platform setting.

VIII. CONCLUSION

We presented VARIBO, a cross-platform query optimizer that replaces the traditional enumerate-and-prune paradigm with a generate-and-explore strategy. VARIBO formulates cross-platform optimization as a structured multiclass classification problem, in which a platform is predicted for each operator of a platform-agnostic plan. At query optimization time, an online tree-convolutional classifier predicts these assignments directly and produces an execution plan without enumerating candidates, avoiding the exponential search space that cross-platform optimization otherwise entails. Offline and in the background, a conditional attribute-regularized β-variational autoencoder coupled with latent-space Bayesian optimization explores alternative plans, executes the most promising ones, and feeds the best back to retrain the classifier. Both components are bootstrapped from an existing cost-based optimizer using only its cost estimates, without executing any plans. Our evaluation shows that VARIBO produces faster execution plans than a cost-based cross-platform optimizer and a learned enumeration-based one, while optimizing queries

about $2\times$ and $14\times$ faster, respectively. Retraining on the explored queries also improves queries that were never explored, and the exploration cost is amortized after a moderate number of workload executions.

Acknowledgment

The work is supported by the Carlsberg Foundation (CF-23-0836). Part of the computation done for this project was performed on the UCloud[6] interactive HPC system, which is managed by the eScience Center at the University of Southern Denmark.

References

[1] I. Giurgiu, S. Barrada, S. Gupta, and M. E. Nidd, "The risk and reward of cross-engine query partitioning: A case for partition-aware benchmarking," in *Proceedings of the 2026 11th International Workshop on Testing Database Systems*, ser. DBTest '26. New York, NY, USA: Association for Computing Machinery, 2026, pp. 1–8. [Online]. Available: https://doi.org/10.1145/3810991.3811629

[2] D. Agrawal, M. Ouzzani, P. Papotti, J.-A. Quiané-Ruiz, N. Tang, S. Thirumuruganathan, A. Troudi, S. Chawla, B. Contreras-Rojas, A. Elmagarmid, Y. Idris, Z. Kaoudi, S. Kruse, J. Lucas, and E. Mansour, "Rheem: enabling cross-platform data processing: may the big data be with you!" *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1414–1427, 2018.

[3] K. Beedkar, B. Contreras-Rojas, H. Gavriilidis, Z. Kaoudi, V. Markl, R. Pardo-Meza, and J. Quiané-Ruiz, "Apache Wayang: A Unified Data Analytics Framework," *SIGMOD Record*, vol. 52, no. 3, pp. 30–35, 2023.

[4] I. Gog, M. Schwarzkopf, N. Crooks, M. P. Grosvenor, A. Clement, and S. Hand, "Musketeer: all for one, one for all in data processing systems," in *Proceedings of the Tenth European Conference on Computer Systems, EuroSys 2015, Bordeaux, France, April 21-24, 2015*, L. Réveillère, T. Harris, and M. Herlihy, Eds. ACM, 2015, pp. 2:1–2:16. [Online]. Available: https://doi.org/10.1145/2741948.2741968

[5] K. Doka, N. Papailiou, V. Giannakouris, D. Tsoumakos, and N. Koziris, "Mix 'n' match multi-engine analytics," in *2016 IEEE International Conference on Big Data (IEEE BigData 2016), Washington DC, USA, December 5-8, 2016*, J. Joshi, G. Karypis, L. Liu, X. Hu, R. Ak, Y. Xia, W. Xu, A. Sato, S. Rachuri, L. H. Ungar, P. S. Yu, R. Govindaraju, and T. Suzumura, Eds. IEEE Computer Society, 2016, pp. 194–203. [Online]. Available: https://doi.org/10.1109/BigData.2016.7840605

[6] V. Josifovski, P. M. Schwarz, L. M. Haas, and E. T. Lin, "Garlic: a new flavor of federated query processing for DB2," in *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data, Madison, Wisconsin, USA, June 3-6, 2002*, M. J. Franklin, B. Moon, and A. Ailamaki, Eds. ACM, 2002, pp. 524–532. [Online]. Available: https://doi.org/10.1145/564691.564751

[7] H. Gavriilidis, K. Beedkar, J. Quiané-Ruiz, and V. Markl, "In-situ cross-database query processing," in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*. IEEE, 2023, pp. 2794–2807. [Online]. Available: https://doi.org/10.1109/ICDE55515.2023.00214

[8] X. Zheng, A. Kumar, and A. Gupta, "Micro: a lightweight middleware for optimizing cross-store cross-model graph-relation joins," in *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montreal, QC, Canada, May 4-8, 2026*. IEEE, 2026, pp. 308–321. [Online]. Available: https://doi.org/10.1109/ICDE65706.2026.00030

[9] Z. She, S. Ravishankar, and J. Duggan, "Bigdawg polystore query optimization through semantic equivalences," in *2016 IEEE High Performance Extreme Computing Conference, HPEC 2016, Waltham, MA, USA, September 13-15, 2016*. IEEE, 2016, pp. 1–6. [Online]. Available: https://doi.org/10.1109/HPEC.2016.7761584

[10] G. X. Yu, Z. Wu, F. Kossmann, T. Li, M. Markakis, A. L. Ngom, S. Madden, and T. Kraska, "Blueprinting the cloud: Unifying and automatically optimizing cloud data infrastructures with BRAD," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3629–3643, 2024.

[6]https://docs.cloud.sdu.dk/

[11] A. Strausz, N. Pardon, and I. Giurgiu, "A learned cost model-based cross-engine optimizer for SQL workloads," *arXiv preprint arXiv:2506.02802*, 2025.

[12] S. Kruse, Z. Kaoudi, B. Contreras-Rojas, S. Chawla, F. Naumann, and J. Quiané-Ruiz, "RHEEMix in the data jungle: a cost-based optimizer for cross-platform systems," *The VLDB Journal*, vol. 29, no. 6, pp. 1287–1310, 2020.

[13] H. Behr, V. Markl, and Z. Kaoudi, "Learn what really matters: A learning-to-rank approach for ML-based query optimization," in *Datenbanksysteme für Business, Technologie und Web (BTW 2023)*, ser. LNI. Gesellschaft für Informatik eV, 2023.

[14] J. Sun and G. Li, "An end-to-end learning-based cost estimator," *Proceedings of the VLDB Endowment*, vol. 13, no. 3, pp. 307–319, 2019.

[15] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proceedings of the 2021 ACM SIGMOD international conference on Management of data*, 2021, pp. 1275–1288.

[16] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019.

[17] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou, "Lero: A learning-to-rank query optimizer," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1466–1479, 2023.

[18] L. Woltmann, J. Thiessat, C. Hartmann, D. Habich, and W. Lehner, "Fastgres: Making learned query optimizer hinting effective," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3310–3322, 2023.

[19] L. Weng, R. Zhu, D. Wu, B. Ding, B. Zheng, and J. Zhou, "Eraser: Eliminating performance regression on learned query optimizer," *Proceedings of the VLDB Endowment*, vol. 17, no. 5, pp. 926–938, 2024.

[20] F. Ventura, Z. Kaoudi, J.-A. Quiané-Ruiz, and V. Markl, "Expand your training limits! generating training data for ml-based data management," in *Proceedings of the 2021 ACM SIGMOD international conference on Management of data*, 2021, pp. 1865–1878.

[21] R. Heinrich, M. Luthra, J. Wehrstein, H. Kornmayer, and C. Binnig, "How good are learned cost models, really? insights from query optimization tasks," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 172:1–172:27, 2025. [Online]. Available: https://doi.org/10.1145/3725309

[22] X. Chen, H. Chen, Z. Liang, S. Liu, J. Wang, K. Zeng, H. Su, and K. Zheng, "LEON: A new framework for ML-aided query optimization," *Proceedings of the VLDB Endowment*, vol. 16, no. 9, pp. 2261–2273, 2023.

[23] Z. Kaoudi, J.-A. Quiané-Ruiz, B. Contreras-Rojas, R. Pardo-Meza, A. Troudi, and S. Chawla, "ML-based Cross-Platform Query Optimization," in *Proceedings of the 36th International Conference on Data Engineering (ICDE)*, 2020, pp. 1489–1500.

[24] A. van Renen, D. Horn, P. Pfeil, K. Vaidya, W. Dong, M. Narayanaswamy, Z. Liu, G. Saxena, A. Kipf, and T. Kraska, "Why TPC is not enough: An analysis of the Amazon Redshift fleet," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3694–3706, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p3694-saxena.pdf

[25] G. H. Bakir, T. Hofmann, B. Schölkopf, A. J. Smola, B. Taskar, and S. V. N. Vishwanathan, *Predicting structured data*. MIT press, 2007.

[26] S. Gulwani, K. K. Mehra, and T. M. Chilimbi, "SPEED: precise and efficient static estimation of program computational complexity," in *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, Z. Shao and B. C. Pierce, Eds., 2009, pp. 127–139.

[27] F. Hueske, M. Peters, M. Sax, A. Rheinländer, R. Bergmann, A. Krettek, and K. Tzoumas, "Opening the black boxes in data flow optimization," *Proceedings of the VLDB Endowment*, vol. 5, no. 11, pp. 1256–1267, 2012.

[28] S. Kruse, Z. Kaoudi, J. Quiané-Ruiz, S. Chawla, F. Naumann, and B. Contreras-Rojas, "Optimizing cross-platform data movement," in *Proceedings of the 35th International Conference on Data Engineering (ICDE)*, 2019, pp. 1642–1645.

[29] L. Mou, G. Li, L. Zhang, T. Wang, and Z. Jin, "Convolutional neural networks over tree structures for programming language processing," in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 1287–1293.

[30] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, 2015, pp. 3483–3491.

[31] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[32] N. Maus, H. Jones, J. Moore, M. J. Kusner, J. Bradshaw, and J. R. Gardner, "Local latent space bayesian optimization over structured inputs," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

[33] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "beta-VAE: Learning basic visual concepts with a constrained variational framework," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=Sy2fzU9gl

[34] A. Pati and A. Lerch, "Attribute-based regularization of latent spaces for variational auto-encoders," *Neural Computing and Applications*, vol. 33, no. 9, pp. 4429–4444, 2021.

[35] O. Shamir, S. Sabato, and N. Tishby, "Learning and generalization with the information bottleneck," *Theor. Comput. Sci.*, vol. 411, no. 29-30, pp. 2696–2711, 2010. [Online]. Available: http://dblp.uni-trier.de/db/journals/tcs/tcs411.html#ShamirST10

[36] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.

[37] E. Begoli, J. Camacho-Rodríguez, J. Hyde, M. J. Mior, and D. Lemire, "Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources," in *Proceedings of the 2018 International Conference on Management of Data*, ser. SIGMOD '18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 221–230. [Online]. Available: https://doi.org/10.1145/3183713.3190662

[38] J. Wehrstein, T. Eckmann, R. Heinrich, and C. Binnig, "JOB-Complex: A challenging benchmark for traditional & learned query optimization," in *Workshop on Applied AI for Database Systems and Applications (AIDB@VLDB)*, 2025, arXiv:2507.07471.

[39] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li, and B. Cui, "Cardinality estimation in DBMS: A comprehensive benchmark evaluation," *Proceedings of the VLDB Endowment*, vol. 15, no. 4, pp. 752–765, 2022.

[40] Transaction Processing Performance Council, "TPC Benchmark H (Decision Support) Standard Specification," https://www.tpc.org/tpch/, 2014, revision 2.18.0.

[41] C. P. Burgess, I. Higgins, A. Pal, L. Matthey, N. Watters, G. Desjardins, and A. Lerchner, "Understanding disentangling in beta-VAE," *arXiv preprint arXiv:1804.03599*, 2018.

[42] J. Petersen, M. S. Pedersen, and Z. Kaoudi, "VariBO: Generate-and-Explore for Learned Cross-Platform Query Optimization," https://github.com/juripetersen/VariBO/blob/main/main.pdf, 2026.

[43] J. V. Szlang, S. Bress, S. Cattes, J. Dees, F. Funke, M. Heimel, M. Oleynik, I. Oukid, and T. Maltenberger, "Workload insights from the snowflake data cloud: What do production analytic queries really look like?" *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5126–5138, 2025.

[44] S. S. Chawathe, H. Garcia-Molina, J. Hammer, K. Ireland, Y. Papakonstantinou, J. D. Ullman, and J. Widom, "The TSIMMIS project: Integration of heterogeneous information sources," in *Proceedings of the 10th Meeting of the Information Processing Society of Japan, Tokyo, Japan, October 1994*, 1994, pp. 7–18.

[45] Z. Yang, W.-L. Chiang, S. Luan, G. Mittal, M. Luo, and I. Stoica, "Balsa: Learning a query optimizer without expert demonstrations," in *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*, 2022, pp. 931–944.

[46] R. Heinrich, X. Li, M. Luthra, and Z. Kaoudi, "Learned cost models for query optimization: From batch to streaming systems," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5482–5487, 2025.

[47] J. Tao, N. Maus, H. Jones, Y. Zeng, J. R. Gardner, and R. Marcus, "Learned offline query planning via bayesian optimization," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–29, 2025.

[48] R. Gómez-Bombarelli, J. N. Wei, D. Duvenaud, J. M. Hernández-Lobato, B. Sánchez-Lengeling, D. Sheberla, J. Aguilera-Iparraguirre, T. D. Hirzel, R. P. Adams, and A. Aspuru-Guzik, "Automatic chemical design using a data-driven continuous representation of molecules," *ACS Central Science*, vol. 4, no. 2, pp. 268–276, 2018.

[49] P. Sulimov, C. Lehmann, and K. Stockinger, "Genjoin: Conditional generative plan-to-plan query optimizer that learns from subplan hints," *Proceedings of the ACM on Management of Data*, vol. 3, no. 4, pp. 247:1–247:25, 2025. [Online]. Available: https://doi.org/10.1145/3749165